

string 정리

문자열을 배열보다 안전하고 편하게 다루는 C++ 표준 문자열 클래스입니다.

사용처 (Where)

- 이름, 문장, 단어를 저장할 때
- 문자열 길이, 검색, 자르기가 필요할 때
- char 배열보다 안전한 문자열 처리가 필요할 때

방법 (How to Use)

- string 변수 선언
- + 또는 append로 문자열 연결
- size(), substr(), find()로 처리
- getline(cin, s)로 공백 포함 입력

기본 정보

- 필요 헤더: #include <string>
- 기본 선언: string s = "hello";

왜 써야 할까? (Why)

- char 배열보다 범위와 길이 관리가 편함
- 입력, 비교, 연결 코드가 짧음
- STL 컨테이너와 함께 사용하기 좋음

주요 함수

함수/문법	의미
size()/length()	문자열 길이
substr(pos, len)	부분 문자열
find(x)	문자 또는 문자열 위치
push_back(c)	문자 추가
empty()	비었는지 확인

기본 코드 예시

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string name = "Kim";
    string msg = name + " studies STL";

    cout << msg << '\n';
    cout << msg.size() << '\n';
    cout << msg.substr(0, 3) << '\n';
}
```

string 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
s[i]	O(1)
find	보통 O(n)
substr	O(len)
문자열 연결	추가 길이에 비례

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
표현력	의도를 타입으로 직접 표현	구조체/클래스를 직접 작성	배열/struct 수동 관리
코드 길이	짧고 표준적인 형태	필드와 생성자 작성 필요	초기화와 접근 실수 가능
유지보수	다른 STL과 자연스럽게 연동	설계가 늘어날수록 관리 필요	가독성이 떨어지기 쉬움
추천 상황	간단한 값 묶음/조건 전달	도메인 의미가 큰 객체	C API와 맞춰야 할 때

응용: 문장 안에서 단어 찾기

find 결과가 string::npos인지 확인하여 단어 포함 여부를 판단합니다.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string text = "I like C++ STL";

    if (text.find("STL") != string::npos) {
        cout << "found";
    }
}
```

처음 배울 때 자주 하는 실수

- cin >> s는 공백 전까지만 입력받음. 문장은 getline 사용
- find 실패 시 string::npos와 비교
- 인덱스 접근 전 size 확인

비슷한 항목과 차이

- `char[]`는 C 스타일이며 직접 널 문자 관리 필요
- `vector<char>`는 문자 배열에 가깝고 문자열 기능은 부족
- `stringstream`은 문자열을 단어/숫자로 파싱할 때 유용

한 줄 정리: `string`은(는) '이름, 문장, 단어를 저장할 때' 상황에서 먼저 떠올리면 좋은 STL입니다.